

Task 1 – Noor Elsharkawy

Theoretical Forensics

1. The “Heavy Class” Myth

1.1 Explain mathematically why the junior developer is wrong.

The developer is wrong because member functions are not stored inside every object. Objects only store their data members.

For example:

```
class Player {  
    int health;  
    int score;  
    void attack();  
};
```

If we create three objects, the memory is approximately:

$3 \times (\text{health} + \text{score}) + \text{one copy of the function code}$

The attack() function is stored only once in memory and is shared by all objects. Therefore, using classes does not make every object store another copy of all its functions.

1.2 Where do the functions live in memory?

The compiled functions live in the **Code/Text segment** of the program's memory. They are not stored inside each individual object.

1.3 How does the compiler know which object the function is modifying?

C++ uses a hidden **this pointer** for non-static member functions.

For example:

```
p1.attack();
```

is conceptually similar to:

```
attack(&p1);
```

The this pointer contains the address of the object that called the function. Therefore, the same function can access and modify the correct object's data.

2. The Memory Map

C++ Element	Memory Location	Explanation
<code>int global_player_count = 0;</code>	Initialized Data	It is a global variable with an initial value.
<code>Enemy* enemy = new Enemy();</code>	Stack + Heap	The pointer itself is on the Stack if it is local, while the Enemy object created by new is on the Heap.
The actual 50MB texture data requested via new	Heap	Memory dynamically allocated using new is stored on the Heap.
The compiled logic for void <code>calculatePhysics()</code>	Code/Text	Functions contain executable instructions stored in the Code/Text segment.
<code>int temp_score;</code> declared inside a while loop	Stack	It is a local variable with automatic storage duration.

Memory Rules

- Functions → **Code/Text**
- Global initialized variables → **Initialized Data**
- Global/static uninitialized variables → **BSS**
- Local variables → **Stack**
- Memory allocated using new → **Heap**

3. The Cliffhanger — Predict Session 2

Manual new and delete can cause memory leaks if an early return or an exception happens before delete is reached.

The C++ feature that solves this problem is **RAII (Resource Acquisition Is Initialization)**, usually implemented using **constructors and destructors**.

A destructor automatically performs cleanup when an object goes out of scope, even if the function exits early or an exception occurs.

Code Review & Bug Hunting

1. Negative dataSize

The critical problem is that the program tries to allocate memory before validating dataSize.

For example, if:

```
dataSize = -5;
```

the program reaches:

```
int* dataBuffer = new int[dataSize];
```

before checking whether the size is valid.

The validation comes too late.

Fix

The program should validate the input before allocating memory:

```
if (dataSize <= 0) {  
    cout << "Invalid request size. Aborting."  
    return;  
}
```

```
int* dataBuffer = new int[dataSize];
```

```
// process the data
```

```
delete[] dataBuffer;
```

This prevents an invalid allocation request and ensures that the allocated memory is released.

2. Nuclear Reactor Safety Check

The class tries to protect `core_temperature` using a private variable and a safe setter. However, the following function creates a security problem:

```
double* getTemperaturePointer() {  
    return &core_temperature;  
}
```

It returns a pointer directly to the private variable.

Anyone who receives this pointer can modify the value without using the safety check in `setTemperature()`.

For example:

```
double* p = reactor.getTemperaturePointer();  
  
*p = 2000.0;
```

This directly changes `core_temperature` and bypasses the safety check.

Why is `delete p` wrong?

`p` points to a member variable inside the object. That memory was not allocated using `new`, so it must **not** be deleted with `delete`.

Fix

The function should not return a modifiable pointer to the private variable. It could return the value itself or a read-only reference/pointer if necessary.

Main issue: Returning a writable pointer breaks the protection provided by private and the safe setter.